



UNIVERSIDADE FEDERAL DE SANTA CATARINA
CENTRO TECNOLÓGICO
DEPARTAMENTO DE INFORMÁTICA E ESTATÍSTICA
CURSO DE GRADUAÇÃO EM CIÊNCIAS DA COMPUTAÇÃO

Estruturas de Dados — INE5408

Rafael Veronezi Ribeiro

Projeto I — Verificação de Cenários e Determinação
de Área de Limpeza por um Robô Aspirador

Florianópolis — 2026

Resumo

Resumo: Relatório técnico do Projeto I de Estruturas de Dados. São documentados dois algoritmos implementados em C++: um validador sintático de XML baseado em Pilha (LIFO) e um algoritmo de determinação de área de componente conexo baseado em Busca em Largura (BFS) com Fila (FIFO). A análise cobre a arquitetura da classe **Cenario** como parser de cursor stateful, a indexação 2D→1D da matriz, a estratégia de marcação antecipada no BFS e a complexidade de ambas as soluções. **Palavras-chave:** Pilha LIFO; Fila FIFO; BFS; Componente Conexo; Parser XML; C++.

Sumário

1	Introdução	2
2	Modelagem do Cenário: classe Cenario	2
2.1	Design de cursor stateful	2
2.2	Pipeline de extração: três métodos em cadeia	2
2.3	Normalização da matriz	3
3	Problema 1: Validação XML com Pilha	3
3.1	Lógica de validação	3
3.2	Diagrama de estados da Pilha	4
4	Problema 2: Determinação de Área por BFS	4
4.1	Localização dos cenários e indexação 2D→1D	4
4.2	Inicialização e condição de partida	4
4.3	Núcleo do BFS: expansão do componente conexo	5
4.4	Simulação do BFS	6
5	Análise de Complexidade	6
6	Conclusão	7
7	Referências	8

1 Introdução

O programa processa arquivos XML que descrevem cenários de operação de um robô aspirador. Dois subproblemas são resolvidos em sequência: (1) validação do aninhamento de tags XML via Pilha, que é pré-requisito de integridade para a etapa seguinte; e (2) determinação da área do componente conexo acessível ao robô em cada cenário, via BFS com Fila. A matriz binária de cada cenário representa o ambiente físico: 1 indica espaço limpável e 0 indica obstáculo. O robô parte de (x, y) e se move nas quatro direções ortogonais (vizinhança-4).

2 Modelagem do Cenário: classe Cenário

2.1 Design de cursor stateful

A classe `Cenario` encapsula o parsing de um único bloco XML. O construtor recebe o texto integral do arquivo e um índice inicial, avança um cursor `pos` progressivamente pelo texto extraíndo cada campo, e ao final armazena em `indice_final` a posição pós-parsing. Esse valor permite ao laço em `main` reiniciar a busca pelo próximo `<cenario>` exatamente onde o parsing anterior terminou, sem reprocessamento.

```
1 Cenario(string &texto, size_t indice_inicial)
2 {
3     size_t pos = indice_inicial;
4     nome     = proxima_tag_conteudo(texto, pos, "nome");
5     altura   = static_cast<size_t>(stoi(proxima_tag_conteudo(texto, pos, "altura"))
6 );
7     largura  = static_cast<size_t>(stoi(proxima_tag_conteudo(texto, pos, "largura")
8 ));
9     x        = static_cast<size_t>(stoi(proxima_tag_conteudo(texto, pos, "x")));
10    y        = static_cast<size_t>(stoi(proxima_tag_conteudo(texto, pos, "y")));
11    matriz    = matriz_remove_espacos(proxima_tag_conteudo(texto, pos, "matriz"));
12    indice_final = pos;
13 }
```

Listing 1: Construtor — parsing sequencial por cursor stateful

2.2 Pipeline de extração: três métodos em cadeia

Cada campo é extraído por três métodos privados que operam sobre o mesmo cursor:

- `proxima_tag`: avança `pos` até encontrar `<`, lê os caracteres até `>` e retorna o identificador da tag. Ao retornar, `pos` aponta para o caractere após `>`.
- `proximo_conteudo`: a partir de `pos` (já após a tag de abertura), acumula caracteres até o próximo `<` (início da tag de fechamento) e avança o cursor além do `>` de fechamento.
- `proxima_tag_conteudo`: chama `proxima_tag` repetidamente até obter o identificador desejado, depois delega a `proximo_conteudo`. É tolerante a tags intermediárias, simplesmente saltando-as.

2.3 Normalização da matriz

```
1 string matriz_remove_espacos(string texto)
2 {
3     string saida;
4     for (size_t i = 0; i < texto.length(); i++)
5         if (texto[i] == '0' || texto[i] == '1')
6             saida += texto[i];
7     return saida;
8 }
```

Listing 2: Filtro de caracteres não numéricos da string da matriz

O XML formatado para leitura humana embute `\n`, `\r` e espaços entre os dígitos da matriz. Sem remover esses caracteres, a fórmula de indexação 2D→1D calcularia deslocamentos incorretos, lendo `'\n'` no lugar de `'0'` ou `'1'` e potencialmente acessando memória fora dos limites da string. O método filtra exclusivamente os caracteres `'0'` e `'1'`, produzindo uma string compacta e indexável.

3 Problema 1: Validação XML com Pilha

3.1 Lógica de validação

O arquivo é carregado integralmente em uma `std::string`. O laço de varredura usa dois índices: `i` percorre o texto e, ao encontrar `<`, um índice auxiliar `j` lê o conteúdo da tag até `>`. A linha `i = j` salta o índice principal para após o `>`, evitando que os caracteres internos da tag sejam reprocessados.

```
1 if (tag_atual[0] == '/') // tag de fechamento
2 {
3     string nome_tag = tag_atual.substr(1);
4     if (pilha_tags.empty() || pilha_tags.top() != nome_tag)
5     {
6         cout << "erro\n";
7         return 0;
8     }
9     pilha_tags.pop();
10 }
11 else // tag de abertura
12     pilha_tags.push(tag_atual);
13
14 // Aps o lao :
15 if (!pilha_tags.empty()) { cout << "erro\n"; return 0; }
```

Listing 3: Núcleo da validação XML: push e pop com verificação de erro

Tags de abertura são empilhadas. Para tags de fechamento, duas condições de erro são verificadas em curto-circuito: pilha vazia (fechamento sem abertura) ou topo diferente do identificador lido (ordem incorreta). Em ambos os casos o programa imprime `"erro"` e executa `return 0` imediatamente, interrompendo toda a execução — nenhum cenário subsequente é avaliado. Após o laço, uma pilha não vazia indica tags sem fechamento correspondente, o terceiro caso de erro, com o mesmo comportamento de encerramento imediato.

3.2 Diagrama de estados da Pilha

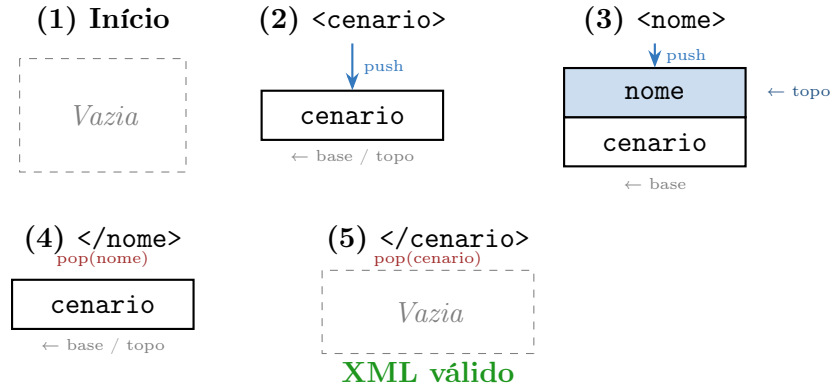


Figura 1: Estados da Pilha durante validação de um fragmento XML bem formado

4 Problema 2: Determinação de Área por BFS

4.1 Localização dos cenários e indexação 2D→1D

Após a validação, o laço em main usa `texto.find(<<"<cenario>", pos)` para localizar cada bloco. A cada iteração, instancia um `Cenario` e atualiza `pos = c.indice_final` para que a próxima busca inicie além do bloco já processado.

A string da matriz (normalizada) armazena os valores em ordem *row-major*. A conversão de coordenadas 2D para índice 1D é:

$$\text{índice} = \text{linha} \times \text{largura} + \text{coluna} \quad (1)$$

Todos os acessos à string da matriz usam essa fórmula: tanto a verificação da posição inicial (`start_idx = c.x * c.largura + c.y`) quanto a verificação dos vizinhos (`n_idx = nx * c.largura + ny`).

4.2 Inicialização e condição de partida

```

1 queue<pair<int,int>> fila;
2 vector<vector<int>> R(c.altura, vector<int>(c.largura, 0));
3 int area = 0;
4 int start_idx = c.x * c.largura + c.y;
5
6 if (c.matriz[start_idx] == '1')
7 {
8     fila.push({c.x, c.y});
9     R[c.x][c.y] = 1;
10    // BFS...
11 }
12 cout << c.nome << " " << area << "\n";

```

Listing 4: Inicialização do BFS para cada cenário

A matriz `R` (visitados) é alocada com dimensões exatas do cenário, inicializada com zeros. O BFS só é executado se a posição inicial for `'1'`; caso contrário, a área impressa é 0, que é o resultado correto para um robô sobre obstáculo.

4.3 Núcleo do BFS: expansão do componente conexo

```
1 int dx[] = {-1, 1, 0, 0};
2 int dy[] = {0, 0, -1, 1};
3
4 while (!fila.empty())
5 {
6     int cx = fila.front().first;
7     int cy = fila.front().second;
8     fila.pop();
9     area++; // contabiliza ao remover
10
11     for (int i = 0; i < 4; i++)
12     {
13         int nx = cx + dx[i];
14         int ny = cy + dy[i];
15
16         if (nx >= 0 && nx < c.altura && ny >= 0 && ny < c.largura)
17         {
18             int n_idx = nx * c.largura + ny;
19             if (c.matriz[n_idx] == '1' && R[nx][ny] == 0)
20             {
21                 fila.push({nx, ny});
22                 R[nx][ny] = 1; // marca ao INSERIR, não ao remover
23             }
24         }
25     }
26 }
```

Listing 5: Laço BFS com vetores de deslocamento estáticos para vizinhança-4

Os arrays `dx` e `dy` codificam os quatro deslocamentos ortogonais (cima, baixo, esquerda, direita), substituindo quatro blocos condicionais por um único `for`. Para cada vizinho, três condições devem ser satisfeitas:

1. **Dentro dos limites:** evita acesso fora da string da matriz.
2. **Valor '1':** o vizinho pertence ao espaço limpável.
3. `R[nx][ny] == 0`: o vizinho ainda não foi enfileirado.

Marcação antecipada: `R[nx][ny] = 1` é executado no momento da inserção na fila, não na remoção. Se fosse feito na remoção, o mesmo nó poderia ser inserido múltiplas vezes por vizinhos distintos que o descobrem antes de ele ser retirado, causando contagem duplicada em `area` e potencial crescimento exponencial da fila em matrizes densas. A marcação antecipada garante que cada célula seja enfileirada no máximo uma vez.

4.4 Simulação do BFS

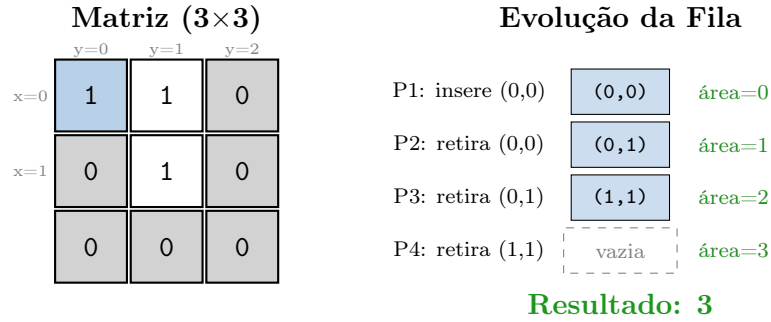


Figura 2: Simulação do BFS em matriz 3×3 com início em (0,0). Células (0,0), (0,1) e (1,1) formam o único componente conexo.

5 Análise de Complexidade

Tabela 1: Complexidade por componente

Componente	Temporal	Espacial
Leitura do arquivo	$O(N)$	$O(N)$
Validação XML	$O(N)$	$O(D)$
Parser Cenário (por bloco)	$O(N_c)$	$O(L_c)$
BFS (por cenário)	$O(V)$	$O(V)$

N : total de caracteres do arquivo; D : profundidade máxima de aninhamento de tags; N_c : caracteres do bloco <cenário>; $L_c = altura \times largura$; $V = L_c =$ número total de células do cenário (área da grade).

Validação XML ($O(N)$): cada caractere é processado no máximo uma vez — o salto $i = j$ impede reproprocessamento de conteúdo interno de tags. Operações de pilha são $O(1)$. O espaço $O(D)$ corresponde à profundidade máxima de aninhamento simultâneo.

BFS ($O(V)$): a marcação antecipada em **R** garante que cada uma das V células seja inserida na fila no máximo uma vez. O laço de 4 vizinhos por célula produz $4V$ verificações no total, mantendo a complexidade temporal linear em V . Quanto ao espaço, a matriz **R** aloca exatamente V inteiros; a fila auxiliar pode conter até V elementos no pior caso (frente de expansão em cenário totalmente limpável); ambas crescem proporcionalmente a $O(V)$ e são liberadas ao fim de cada cenário (escopo de bloco).

6 Conclusão

O projeto demonstrou que estruturas de dados lineares, em particular Pilha e Fila, oferecem abstrações precisas e eficientes para classes importantes de problemas computacionais: o controle de contexto aninhado e a exploração de componentes conexos, respectivamente.

A principal contribuição de engenharia do projeto reside no design da classe `Cenario` como um parser de cursor stateful. A decisão de encapsular os métodos `proxima_tag`, `proximo_conteudo` e `proxima_tag_conteudo` em uma hierarquia privada permitiu separar claramente as responsabilidades de navegação no texto (localização de delimitadores) e de extração de valores (leitura de conteúdo), tornando o código do construtor linear e legível. O mecanismo de `indice_final` garante que a iteração sobre múltiplos cenários seja eficiente, sem redundância de processamento.

A **difículdade central** do desenvolvimento foi a normalização da string da matriz. O método `matriz_remove_espacos` foi necessário porque o XML, ao ser formatado visualmente para legibilidade humana, inclui quebras de linha e espaços entre os dígitos da matriz. Sem esse passo, a fórmula de indexação $linha \times largura + coluna$ produziria deslocamentos incorretos, resultando em leitura de caracteres errados (como `'\n'` em vez de `'0'` ou `'1'`) e potencialmente em acesso fora dos limites da string. A solução de filtrar exclusivamente `'0'` e `'1'` é simples, robusta e independente do formato específico de whitespace usado no arquivo.

A estratégia de marcação antecipada no BFS (`R[nx][ny] = 1` na inserção) é outro ponto de atenção que exigiu cuidado: uma implementação ingênua que marcasse na remoção permitiria duplicatas na fila, corrompendo a contagem de área em matrizes densas

7 Referências

1. DROZDEK, Adam. **Data Structures and Algorithms in C++**. 4th ed. Boston: Cengage Learning, 2013. ISBN: 978-1133608424.
2. CPPREFERENCE CONTRIBUTORS. **C++ Reference - Standard Library Documentation**. 2026. Disponível em: <<https://en.cppreference.com/>>. Acesso em: 13 mai. 2026.
3. GEEKSFORGEEKS. **Breadth First Search or BFS for a Graph**. 2024. Disponível em: <<https://www.geeksforgeeks.org/breadth-first-search-or-bfs-for-a-graph/>>. Acesso em: 13 mai. 2026.
4. UFSC. **INE5408 - Estruturas de Dados: Materiais didáticos e notas de aula**. Plataforma Moodle , CTC/INE/UFSC, 2026.